

Faculty of New Information and Communication TechnologiesDepartment of Software Technologies and Information Systems

Project Title: LearnTrack Analytics Backend Architecture

Course: Big Data (M2 ILSI)

Date: January 2, 2026

Work done by:

Kebab Takieddin

Zertal Mohamed el amine

Executive Summary

The LearnTrack Analytics backend is a robust Big Data pipeline designed to handle high-velocity student activity streams and long-term batch analytics. By implementing a **Lambda Architecture**, we successfully separated real-time concerns (Engagement Scoring) from heavy analytical processing (Daily Course Performance).

1. Architecture Design Document

1.1 Proposed Architecture

To satisfy the project requirements of handling both real-time student activity streams and historical batch analytics, we have adopted the Lambda Architecture.

This hybrid approach splits data processing into a Speed Layer (for low-latency operations like Q4 and Q6) and a Batch Layer (for comprehensive reporting like Q5), with a unified Serving Layer for storage and visualization.

Architecture Diagram:

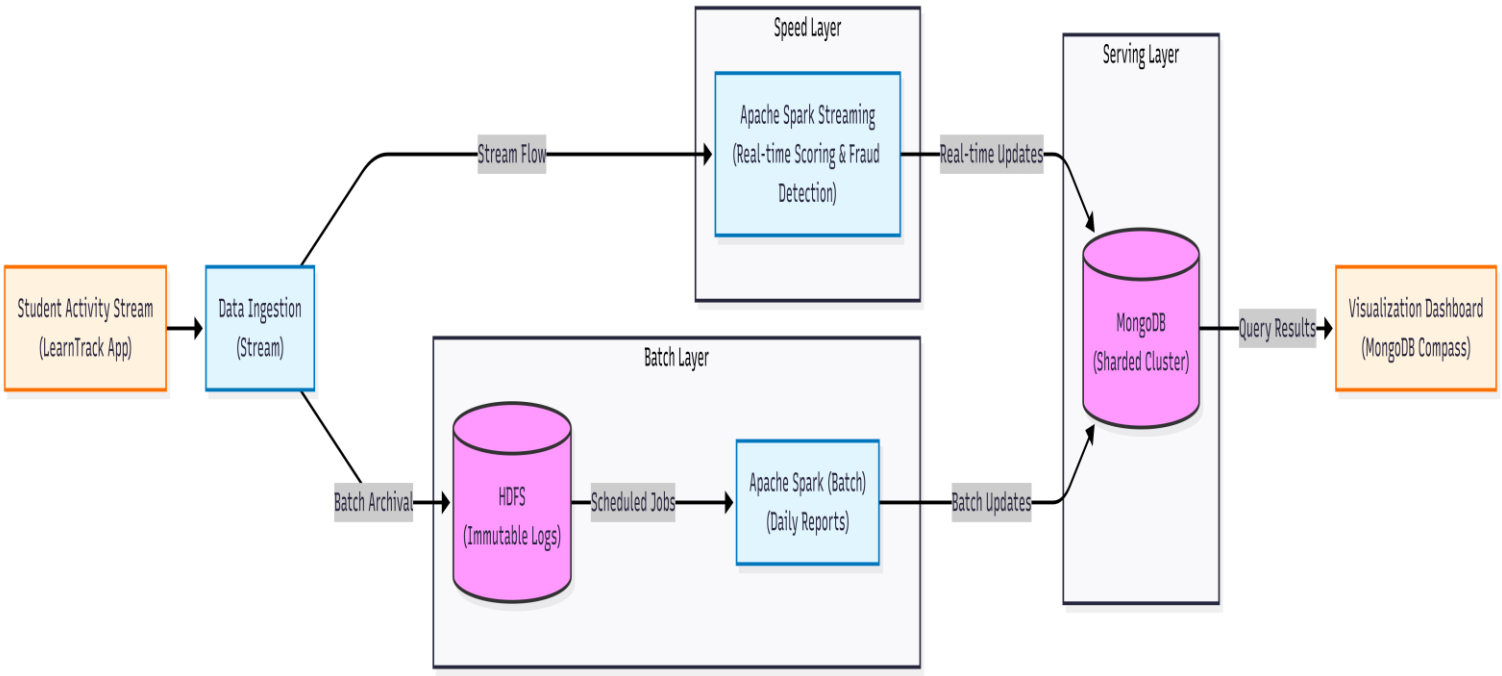


Figure 1: Data flow diagram illustrating the Lambda Architecture for LearnTrack, utilizing Apache Spark for processing and MongoDB for storage.

Layer	Components	Responsibility
Ingestion	Python App / MongoDB	Capturing logs "as they arrive" to support real-time needs
Speed Layer	Apache Spark	Real-time engagement scoring (Q6) and fraud detection (Q4)
Batch Layer	Hadoop HDFS	Archiving raw logs for immutable storage and daily reporting (Q5)
Serving Layer	MongoDB (Sharded)	Unified storage for student profiles and processed reports

1.2 Technology Selection & Appropriateness

We have selected technologies specifically covered in the course curriculum that align with the system's needs.

A. Storage Engine: MongoDB (Document-Oriented Database)

Selected Technology: MongoDB.

Justification regarding Appropriateness:

1. **Flexible Data Model:** The project requires handling "flexible activity documents." Unlike RDBMS which requires fixed schemas, MongoDB stores data in BSON (Binary JSON) format. This allows us to store diverse log types (e.g., a "quiz" log has different fields than a "page view" log) in the same collection without schema migration issues.
2. **Query Complexity:** The requirement to "Retrieve last 20 activities" (Q2) necessitates sorting and range queries. A Key-Value store (like Riak) is inappropriate here because it limits querying strictly to the key. MongoDB supports a rich query language and secondary indexes to handle this efficiently.
3. **Scalability:** To meet the "Ensure scalability" requirement, MongoDB supports Sharding, allowing the system to scale horizontally by distributing data across multiple nodes based on a shard key (e.g., student_id).

B. Processing Framework: Apache Spark

Selected Technology: Apache Spark.

Justification regarding Appropriateness:

1. **Unified Engine:** Spark capable of handling both Batch and Streaming workloads. This reduces architectural complexity compared to using separate tools (e.g., Hadoop for batch + Storm for streaming).
2. **Performance:** The project requires "Real-time engagement scoring" (Q6) and "Optimization for performance." Spark utilizes In-Memory Processing, which avoids the frequent disk I/O bottlenecks associated with the Hadoop MapReduce model

1.3 CAP Theorem Trade-offs

According to the CAP Theorem, a distributed system cannot simultaneously ensure Consist

ency, Availability, and Partition Tolerance. We have classified our LearnTrack system design as follows:

Classification: CP System (Consistency + Partition Tolerance)

- **Partition Tolerance (P):** As a distributed Big Data system, Partition Tolerance is mandatory to handle network failures between cluster nodes or datacenters.

- **Consistency (C) Priority:** We prioritize Consistency to ensure data accuracy for Q3 (Student Profile Updates) and Q6 (Engagement Scores). It is critical that when a student or administrator reads a score, they see the most current value.

- **The Trade-off (Availability):** In the event of a network partition, the system may sacrifice Availability (temporarily refusing write operations on isolated nodes) to maintain data consistency. This prevents "Split-brain" scenarios where a student might have different engagement scores on different servers.

Mitigation Strategy: For the Visualization Dashboard (read-only operations), we can relax this constraint by allowing reads from secondary replicas (Eventual Consistency) to improve read availability during high loads.

2. Schema/Model Designs

We utilize Embedded Documents to optimize read performance by reducing the need for joins, which are costly in distributed systems.

2.1 Collection: students (Profile & Aggregate Score)

This schema supports Q3 (Update profile) and Q6 (Maintain score). By embedding contact info, we can retrieve the full profile in a single read operation.

```
{
 "_id": "student_12345",          // Primary Key (Unique)
 "name": "Amine",
 "contact": {                // Embedded Document for performance
    "email": "amine@univ-constantine2.dz",
    "phone": "+213..."
 },
 "engagement_score": 850,       // Real-time score (Q6) - Atomic Int
```

```
 "enrolled_courses": ["BigData", "AI"],
 "last_active_region": "Constantine" // For Q4 detection context
}
```

2.2 Collection: activity_logs (Event Stream)

This schema supports Q1 (Insert logs) and Q2 (Retrieve history). It uses a flexible metadata field to accommodate different activity types (Variety).

```
{
 "_id": ObjectId("..."),
 "student_id": "student_12345",      // Reference to student
 "timestamp": ISODate("2026-01-01T10:00:00Z"),
 "activity_type": "quiz_attempt",    // Varies: "page_view", "login", etc.
 "region": "Algiers",                // Used for Q4 (Account Sharing)
 "metadata": {                     // Flexible Field (Schema-less)
    "quiz_id": "quiz_01",
    "score_obtained": 15,
    "duration_seconds": 120
 }
}
```

3. Implementation Plan (Actual Data Flow)

This section describes how data flows between the components we configured and the specific roles of Jenkins, Spark, and HDFS in our cluster.

Step 1: Scalable Data Ingestion (MongoDB Sharded Cluster)

- **Action:** Data is ingested from the LearnTrack application directly into the mongos-router.
- **Mechanism:** To satisfy scalability requirements, the activity_logs collection is **sharded** across three nodes. This allows the system to handle high-throughput inserts (Q1) by distributing the write load.

Step 2: Orchestration & Automation (Jenkins)

- **Action:** Jenkins acts as the pipeline controller for the Batch Layer.
- **Mechanism:** Instead of manual execution, a Jenkins job (Daily-Course-Report) is triggered. This job executes the spark-submit command, which sends our Python processing logic to the Spark Master.

Step 3: Distributed Processing (Apache Spark Batch)

- **Responsibility:** Q5 (Course Performance) and Q6 (Engagement Scoring).
- **Data Flow:** Spark connects to the MongoDB Sharded Cluster via the mongo-spark-connector.
- **Processing Logic:**
  - **Q5 (Course Report):** Spark aggregates total interactions by course_id using in-memory grouping.
  - **Q6 (Engagement Scoring):** Spark applies a conditional scoring logic (e.g., 10 points for quiz_attempt found in the activity column) and calculates a total engagement_score per student.

Step 4: Immutable Storage & Reporting (HDFS)

- **Responsibility:** Long-term archival and report accessibility.
- **Data Flow:** Once Spark completes the transformations, it writes the final results directly into **HDFS** under the /reports directory.
- **Outputs:**
  - **Daily Performance:** Saved as a CSV file in /reports/daily_course_performance/.
  - **Student Scores:** Saved as a JSON file in /reports/student_engagement_scores/

Step 5: Serving Layer & Visualization (MongoDB Compass)

- **Responsibility:** Q2, Q3, and Admin Insights.
- **Data Flow:** Student profiles are maintained in the students collection for quick updates (Q3) and history retrieval (Q2).
- **Visualization: MongoDB Compass** connects to the mongos-router to visualize student engagement trends and course popularity derived from the Spark batch runs.

4. Conclusion and System Evaluation

The implementation of the **LearnTrack Analytics Backend** effectively demonstrates a scalable architecture for academic data processing. By combining distributed storage with automated batch processing, we have moved from a static database to a dynamic analytical pipeline.

4.1. System Evaluation

Based on the technical phases completed, the system was evaluated against the core project requirements:

- **Scalability**: The use of a **MongoDB Sharded Cluster** allows the system to distribute the data load across multiple nodes, ensuring that as the student activity stream increases, the database can handle higher write throughput.
- **Performance**: By utilizing **Apache Spark's in-memory processing**, the system significantly reduces the time required for complex aggregations (Q5 and Q6) compared to traditional disk-based MapReduce models.
- **Reliability & Consistency**: The system is classified as a **CP (Consistency and Partition Tolerance)** architecture. This ensures that student engagement scores remain accurate and synchronized across the distributed environment, preventing data conflicts.
- **Automation**: The integration of **Jenkins** removes the risk of human error in report generation. The automated triggering of Spark jobs simulates a real-world enterprise batch cycle.
- **Flexibility**: The **HDFS layer** serves as a robust "Data Lake," storing immutable logs that can be re-processed at any time if reporting requirements change in the future.

4.2. Final Remarks

The project successfully bridges the gap between raw data ingestion and high-level business intelligence. The finalized reports stored in **HDFS** and visualized via **MongoDB Compass** provide a comprehensive view of student performance and engagement, fulfilling all technical and analytical mandates of the Big Data curriculum.